

Enum, Stream and Collections — A Readable Companion to the Elixir Docs

Chapter 6 · checked against Elixir 1.20 · examples run on Elixir 1.14 · Sep 26, 2026

1. Why collections are where performance hides

Most Elixir code is pipelines over collections: `Enum.map`, `Enum.filter`, `Enum.reduce`. They read the same whether the data has ten elements or ten million, which is exactly the problem. A line that is instant in a test can be quadratic in production, and nothing in the syntax tells you.

Two facts explain nearly every collection bug in Elixir:

1. **Lists are singly linked.** Reaching the front is instant; reaching the n th element, the end, or the length means walking the whole list.
2. **Enum is eager and Stream is lazy.** Every `Enum` step builds a full intermediate result. A `Stream` builds a recipe that runs only when something consumes it, and runs again every time something consumes it.

2. The building blocks

Cost of common operations on a list

Operation	Cost	Note
Prepend, <code>hd/1</code> , head-tail pattern match	Constant	Always prefer working at the front
<code>length(list)</code> , <code>Enum.count(list)</code>	Linear	Don't use to test for emptiness
<code>Enum.at(list, i)</code>	Linear in <code>i</code>	Lists are not arrays
<code>list ++ other</code>	Linear in <code>list</code>	Copies the left side
<code>List.last(list)</code>	Linear	
<code>Enum.reverse(list)</code>	Linear, one pass	Cheap way to finish a prepend-based build

For indexed access use a tuple (`elem/2`) or a map keyed by index; for sets use `MapSet`; for queues use `:queue`.

Enum or Stream?

Use Enum when	Use Stream when
The data fits comfortably in memory	The data is large or unbounded

Use Enum when	Use Stream when
You need the whole result	You need only part of it (<code>Enum.take/2</code> , <code>Enum.find/2</code>)
The default	Reading from slow sources: files (<code>File.stream!/1</code>), sockets, paginated APIs

The docs recommend starting with `Enum` and moving to `Stream` only when “laziness is required.”

Comprehensions and early exit

- `for x <- xs, filter, into: %{} build`s any collectable; `reduce`: turns it into a fold.
- `Enum.reduce_while/3` stops as soon as the function returns `{:halt, acc}`.
- `Enum.find/2`, `Enum.any?/2` and `Enum.take/2` stop early too.

3. How it actually works

Streams are recipes, not data

A `Stream` value holds the source and the chain of functions. Each `Enum` call on it runs the whole chain from the start. Side effects in the chain (logging, HTTP calls, counters, reading a file) happen again every time the stream is consumed.

Maps are not ordered

Maps with up to 32 keys happen to iterate in sorted key order; larger maps use a hash structure with no useful order. Neither follows insertion order. Code that relies on map order works by accident, and breaks when the map grows past 32 keys or the keys change.

Ranges pick their own direction

`first..last` without an explicit step counts *down* when `first > last`. The docs call this behaviour deprecated and recommend explicit steps: `1..n//1` is empty when `n` is 0, and `3..1//-1` for real decreasing ranges.

4. Plausible but wrong

4.1 Indexing a list in a loop

```
for i <- 0..(length(items) - 1) do
  process(Enum.at(items, i), i)
end
```

Each `Enum.at/2` walks from the start, so the loop is quadratic. With 20,000 items, compiled code took **356 ms**, against **1.2 ms** for `Enum.map/2` over the same list. Fix: `Enum.with_index(items)` and iterate once:

```
for {item, i} <- Enum.with_index(items), do: process(item, i)
```

4.2 Appending in a reduce

```
Enum.reduce(rows, [], fn row, acc -> acc ++ [transform(row)] end)
```

`++` copies the accumulator on every step. For 20,000 rows this took **842 ms**, against **0.2 ms** for prepending and reversing once (measured with compiled code). Fix: `Enum.map(rows, &transform/1)`, or prepend with `[x | acc]` and call `Enum.reverse/1` at the end.

4.3 length for emptiness

```
if length(queue) > 0, do: dispatch(queue)
```

This counts every element to answer a yes-or-no question. On a long queue checked often it becomes a hot spot. Fix: `queue != []`, or match in a function head: `def dispatch([_ | _] = queue).`

4.4 A range that runs backwards

```
def retry(n) do
  for attempt <- 1..n, do: try_once(attempt)
end
retry(0)  # runs attempts 1 and 0
```

With `n = 0` the range is `1..0`, which counts down, so the loop runs twice instead of not at all (verified: `[1, 0]`). Off-by-one bugs like this are easy to miss because they only show at the boundary. Fix: `1..n//1`, which is empty when `n < 1`.

4.5 A stream consumed twice

```
events = Stream.map(ids, &Api.fetch_event!/1)
total = Enum.count(events)
first_ten = Enum.take(events, 10)
```

The stream runs once per consumer: every event is fetched for `count`, then the first ten are fetched again (verified: side effects ran twice). With a file stream, the file is re-read; with a paginated API, you pay twice and may get different data. Fix: materialise once with `Enum.to_list/1` when the data fits, or restructure into a single pass.

4.6 Relying on map order

```
headers = Map.keys(row) |> Enum.join(",")
values = Map.values(row) |> Enum.join(",")
```

The column order is sorted, not the order you wrote the keys (verified: `%{"zeta" => 1, "alpha" => 2}` gives `alpha` first). Once a row has more than 32 fields, the order is effectively arbitrary. Fix: keep an explicit list of columns and read values with `Map.fetch!/2` in that order, or use a keyword list when order is part of the data.

4.7 Loading everything to use a little

```
File.read!("access.log")
|> String.split("\n")
|> Enum.filter(&String.contains?(&1, "500"))
|> Enum.take(5)
```

For a multi-gigabyte log this reads and splits the whole file into memory to return five lines. Fix: `File.stream!("access.log") |> Stream.filter(...) |> Enum.take(5)` reads line by line and stops after the fifth match.

5. Review checklist and sources

- ☐ No `Enum.at/2` inside loops over lists; `Enum.with_index/1` or a different structure instead
- ☐ No `acc ++ [x]` in reductions; prepend and reverse, or use `Enum.map/2`
- ☐ Emptiness is tested with `== []` or pattern matching, not `length/1`
- ☐ Ranges that can be empty use an explicit step (`1..n//1`)
- ☐ Streams with side effects are consumed exactly once
- ☐ No code depends on map iteration order
- ☐ Large or slow sources are processed with `Stream` and stop early where possible
- ☐ `Enum` is the default; `Stream` is used for a reason

Sources

- Enumerables and streams: <https://hexdocs.pm/elixir/enumerable-and-streams.html>
- Enum: <https://hexdocs.pm/elixir/Enum.html>
- Stream: <https://hexdocs.pm/elixir/Stream.html>
- Range: <https://hexdocs.pm/elixir/Range.html>
- List: <https://hexdocs.pm/elixir/List.html>